Project Title: A simple library

Objective: The objective of this project is to design and implement the API for a simple library system, as described below.The implementation will incorporate resource-oriented API design patterns as studied in classes.

Project Description: In this project, students will work in groups of 2 to create a library management system. This API is based on the management of library, book, and person resources. Each library has several book copies, each book copy exists in a single library and people can loan books from libraries they are registered in.

Project Requirements:

- Standard methods should exist for all relevant resources. All resources should be considered static (no fields can be added or removed).
- Listing operations can be filtered by user name, book author name, or library location. Pagination should be used to ensure no more than 10 results are returned to the user in each single response.
- The API should allow users to register into libraries, cancel their registration into a library, loan books from a library they are registered in, and return books they have previously loaned.
- The API should also allow the following operations: obtaining the historic list of books loaned by a user, the current list of books loaned by a user and moving a book copy to another library (you are free to use your discretion to identify, solve and/or ignore any problems this operation might entail).
- Field masks should be implemented for relevant standard operations.
- The full text or an extended overview is available as a singleton sub-resource of a book resource. This defaults to no text (empty text).
- The system allows the API user to search the fulltext of books in all libraries for a specific word. This is a long running operation. (make sure you find solutions that allow testing/debugging the correct operation of the LRO).

The above requirements imply the use of studied patterns such as field masks, standard and custom methods, pagination, move operation, singleton sub-resources, long running operations and some approach to deal with many-to-many associations (e.g. library / user). Feel free to use other approaches you find to be useful.

Deliverables and submission: You should submit your code, no later than 12:00 19th of December, in the link provided in moodle. You should prepare a Postman project to test/demonstrate the functionality of the system.

Defense: The project will be subjected to mandatory discussion/defense during class on the 19th December. There will be registration for time slots allocated for this defense.

You will be required to demonstrate the functionality and present and discuss design patterns use in this project. You might also be required to discuss the implementation of potential changes or evolutions of the system, with respect to API design pattern use.

You are free to use whatever resources you want to support your development, but you are assumed to have complete ownership of the codebase you submit, so you are expected to be able to explain, change and discuss all parts of your submission with ease.

If you cannot make it to class on that date, please get in touch with the Professor so that alternative arrangements can be made. Nevertheless, you should still submit your work by the due date.